

AI Prompt: Cost-Optimised OpenAI Model Router

Objective

Build an OpenAI model router that minimises API cost while maintaining quality.

Principles

1. Default to the cheapest model.
2. Escalate only when the task genuinely requires stronger reasoning.
3. Never use the most expensive model as the default.
4. Keep model names configurable through environment variables.
5. The routing logic belongs in application code, not in environment variables.

Environment Variables

```env

OPENAI\_MODEL\_DEFAULT=gpt-5-nano

OPENAI\_MODEL\_PREMIUM=gpt-5-mini

OPENAI\_MODEL\_REASONING=gpt-5.6

```

Architecture

```

User Request

■

▼

Task Classifier

■

■■■ Simple Chat

■■■ Search

■■■ Extraction

■■■ Writing

■■■ Coding

■■■ Deep Reasoning

■

▼

Model Router

■

■■■ gpt-5-nano

■■■ gpt-5-mini

■■■ gpt-5.6

```

JavaScript Example

```
```javascript
const MODELS = {
 DEFAULT: process.env.OPENAI_MODEL_DEFAULT,
 PREMIUM: process.env.OPENAI_MODEL_PREMIUM,
 REASONING: process.env.OPENAI_MODEL_REASONING,
};

function chooseModel(task) {
 switch (task.type) {
 case "chat":
 case "faq":
 case "search":
 case "telegram":
 case "json":
 case "classification":
 return MODELS.DEFAULT;
 case "summarise":
 case "rewrite":
 case "translation":
 case "bible-study":
 case "email":
 return MODELS.PREMIUM;
 case "coding":
 case "architecture":
 case "deep_reasoning":
 case "financial_analysis":
 return MODELS.REASONING;
 default:
 return MODELS.DEFAULT;
 }
}
```
```

Calling OpenAI

```
```javascript
const model = chooseModel(task);
const response = await client.responses.create({
```

```

model,
input: userPrompt
});
...

Automatic Escalation
```javascript
function requiresReasoning(task) {
  return (
    task.requiresMultipleSteps ||
    task.hasLargeCodebase ||
    task.hasManyDocuments ||
    task.userRequestedExpertMode
  );
}
const model = requiresReasoning(task)
  ? MODELS.REASONING
  : chooseModel(task);
...

---

## Recommended Routing
| Task | Model |
|-----|-----|
| Greeting | gpt-5-nano |
| FAQ | gpt-5-nano |
| Search | gpt-5-nano |
| Extraction | gpt-5-nano |
| Email | gpt-5-mini |
| Summarisation | gpt-5-mini |
| Bible Study | gpt-5-mini |
| Code Review | gpt-5.6 |
| Architecture | gpt-5.6 |
| Complex Analysis | gpt-5.6 |
...

## Production Recommendations
- Target ~98% of requests to `gpt-5-nano`.
- Escalate to `gpt-5-mini` for moderate reasoning or higher-quality writing.
- Reserve `gpt-5.6` for rare, high-value tasks requiring deep reasoning.

```

- Log the selected model and token usage for every request.
- Set per-request `max_output_tokens` to avoid excessive output costs.
- Keep prompts concise and avoid resending unnecessary conversation history.